Chapter 7 | Worked solutions with Python

GPT, BERT, and T5

All fourteen exercises are solved in order. The first seven use small arrays, diagrams, and calculations. The later experiments trace actual corruption decisions, distinguish direct loss gradients from context gradients, check information paths, fit the three prepared examples, and adapt a classifier. The last exercise specifies a fair downstream evaluation.

Keep three questions separate: what enters the network, what each position can read, and which predictions are graded. Shared Transformer blocks can solve different learning problems when those choices change. The written edition includes every explanation and measured result; the Python edition also includes the code and saved outputs.

Run this notebook inside `transformers-book` with `requirements-ch05.txt` and Jupyter installed. Restart the kernel and run all cells. The small CPU experiments use locally constructed data. Fixed seeds make comparisons repeatable in the recorded environment; other numerical versions can change training trajectories and selected checkpoints. No canonical source is edited.

In [1]

```
from pathlib import Path
import sys
import math, platform
from copy import deepcopy

import torch
from torch import nn
from torch.nn import functional as F

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch07_pretraining" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )

torch.set_num_threads(1)
torch.use_deterministic_algorithms(True)
from examples.ch07_pretraining.data import (
    VOCAB, PAD, BOS, EOS, MASK, IGNORE, TOKEN_TO_ID,
    causal_pairs, bert_corrupt, span_corrupt, encode, token_names,
)
from examples.ch07_pretraining.model import TinyTokenModel, selected_loss
print("Python", platform.python_version(), "| PyTorch", torch.__version__)
print("Device: CPU | threads:", torch.get_num_threads())
```

Output

```
Python 3.12.14 | PyTorch 2.15.0
Device: CPU | threads: 1
```

1. Move the answer one place ahead

From `the red car stopped`, retain the first three words as inputs and the last three as labels. Read each row as one question: after the available prefix, which word occurred next?

Position	Input	Readable prefix	Reference label
0	the	the	red
1	red	the red	car
2	car	the red car	stopped

The diagonal gives position 1 access to its current input `red`, while the answer `car` is at the later input position 2. That later key is blocked. The separate label array enters the loss after the logits are computed; it supplies no attention keys or values. Every layer must preserve this rule, so a future token cannot take an indirect route through another position.

For `the old car moved today`, inputs are `the old car moved` and labels are `old car moved today`. The readable prefixes have lengths 1, 2, 3, and 4. The first word is unscored starting context in this example, which uses no BOS or EOS. In general, N words produce $N - 1$ shifted input-label pairs.

In [2]

```
for sentence in ("the red car stopped", "the old car moved today"):
    inputs, labels = causal_pairs(encode(sentence))
    print(sentence)
    for position, label in enumerate(labels):
        prefix = " ".join(token_names(inputs[:position + 1]))
        print(f"  position {position}: {prefix:20s} -> {VOCAB[label]}")
```

Output

```
the red car stopped
  position 0: the                  -> red
  position 1: the red              -> car
  position 2: the red car          -> stopped
the old car moved today
  position 0: the                  -> old
  position 1: the old              -> car
  position 2: the old car          -> moved
  position 3: the old car moved    -> today
```

2. Draw the information paths

Rows are queries: the positions asking for information. Columns are keys: the positions available to those queries. A 1 permits a read; a 0 blocks it.

Query	Causal keys 0, 1, 2, 3	Bidirectional keys 0, 1, 2, 3
0	1 0 0 0	1 1 1 1
1	1 1 0 0	1 1 1 1
2	1 1 1 0	1 1 1 1
3	1 1 1 1	1 1 1 1

Causal attention permits $1 + 2 + 3 + 4 = 10$ cells; bidirectional attention permits $4 \times 4 = 16$. In cross-attention, the three queries belong to the decoder and the four keys to the encoder. All three rows are `1 1 1 1`, for 12 cells. The two sequence lengths are independent. A decoder query at position 0 may read encoder key 3: the decoder's causal rule applies to decoder self-attention, not to a comparison between source and target indices.

If encoder key 3 is padding, every cross-attention row becomes `1 1 1 0`. The shape stays 3×4, but only nine cells remain permitted. Attention weights normalize across the remaining valid keys in each row.

In [3]

```
causal_grid = torch.ones(4, 4, dtype=torch.bool).tril()
bidirectional_grid = torch.ones(4, 4, dtype=torch.bool)
cross_grid = torch.ones(3, 4, dtype=torch.bool)
padded_cross_grid = cross_grid & torch.tensor([True, True, True, False])
for name, grid in (("Causal", causal_grid),
                   ("Bidirectional", bidirectional_grid),
                   ("Cross", cross_grid), ("Cross with PAD", padded_cross_grid)):
    print(name, "shape", tuple(grid.shape), "permitted", grid.sum().item())
    for row in grid: print(" ", " ".join(str(int(value)) for value in row))
```

Output

```
Causal shape (4, 4) permitted 10
  1 0 0 0
  1 1 0 0
  1 1 1 0
  1 1 1 1
Bidirectional shape (4, 4) permitted 16
  1 1 1 1
  1 1 1 1
  1 1 1 1
  1 1 1 1
Cross shape (3, 4) permitted 12
  1 1 1 1
  1 1 1 1
  1 1 1 1
Cross with PAD shape (3, 4) permitted 9
  1 1 1 0
  1 1 1 0
  1 1 1 0
```

3. Separate selection from replacement

First choose the positions to grade. Then choose what the model sees at each selected position. Applying the 15% rate to 200 eligible tokens gives 30 selected positions in expectation. The 80/10/10 mixture is conditional on that selection, not a percentage of all 200 tokens.

Decision	Expected count among 200 eligible tokens
Selected and replaced by MASK	$200(0.15)(0.8) = 24$
Selected and given a random ordinary word	$200(0.15)(0.1) = 3$
Selected and unchanged	$200(0.15)(0.1) = 3$
Not selected	$200(0.85) = 170$

All 30 selected positions have their original token as a label, including the three unchanged ones. Only 12% of all eligible inputs are expected to become MASK. The counts describe expected rates; an implementation may round or fix a selected count per example, while the sampled treatments vary.

In this teaching vocabulary the random branch chooses among ten ordinary words, including the original word. Conditional on that branch, the chance of drawing the original again is $1/10$. Consequently the input ID alone cannot identify which treatment was sampled.

In [4]

```
eligible, selection_rate = 200, 0.15
selected_count = eligible * selection_rate
print("Expected selected / graded labels:", selected_count)
for name, fraction in (("mask", .8), ("random", .1), ("unchanged", .1)):
    print(name, "expected count:", selected_count * fraction)
print("Not selected:", eligible - selected_count)
print("MASK fraction of all eligible tokens:", selection_rate * .8)
```

Output

```
Expected selected / graded labels: 30.0
mask expected count: 24.0
random expected count: 3.0
unchanged expected count: 3.0
Not selected: 170.0
MASK fraction of all eligible tokens: 0.12
```

4. Keep three masks separate

For the supplied sequence `the [MASK] car stopped PAD`, the key-validity row is `1 1 1 1 0`. The labels are `IGNORE red IGNORE IGNORE IGNORE`. The selected query can read MASK: it is a real vocabulary token with an embedding. It cannot read the PAD key. Input replacement changes the contents, key validity changes communication permissions, and loss selection changes which output rows are graded.

Only one label contributes to the loss, so a correct-token probability of 0.8 gives the loss sum and mean

$$L_{\text{sum}} = -\log(0.8) \approx 0.223144, \qquad L_{\text{mean}} = L_{\text{sum}}/1 \approx 0.223144.$$

Dividing by the four valid inputs would give 0.055786; dividing by the five stored positions would give 0.044629. Both dilute the penalty with positions that have no prediction target. Context may help a prediction without contributing an extra term to its loss.

In [5]

```
mask_inputs = torch.tensor([[TOKEN_TO_ID["the"], MASK,
                             TOKEN_TO_ID["car"], TOKEN_TO_ID["stopped"], PAD]])
mask_labels = torch.tensor([[IGNORE, TOKEN_TO_ID["red"], IGNORE, IGNORE, IGNORE]])
probabilities = torch.full((1, 5, len(VOCAB)), 1 / len(VOCAB), dtype=torch.float64)
probabilities[0, 1] = .2 / (len(VOCAB) - 1)
probabilities[0, 1, TOKEN_TO_ID["red"]] = .8
mask_loss = selected_loss(probabilities.log(), mask_labels).item()
print("Available keys:", (mask_inputs != PAD).int().tolist())
print("Labels:", token_names(mask_labels[0]))
print(f"Correct mean {mask_loss:.6f}; divided by four {mask_loss/4:.6f}; by five {mask_loss/5:.6f}")
assert math.isclose(mask_loss, -math.log(.8), abs_tol=1e-12)
```

Output

```
Available keys: [[1, 1, 1, 1, 0]]
Labels: ['-', 'red', '-', '-', '-']
Correct mean 0.223144; divided by four 0.055786; by five 0.044629
```

5. Put the fragments back in order

Zero-based indices 0 through 8 correspond to `the red car stopped near the old bridge today`. The half-open intervals `(1, 3)` and `(6, 8)` remove `red car` and `old bridge`, respectively. Each interval includes its start and excludes its end.

Array	Tokens
Source	`the S0 stopped near the S1 today EOS`
Target	`S0 red car S1 old bridge S2 EOS`
Decoder input	`BOS S0 red car S1 old bridge S2`

All three arrays have eight tokens. The source already supplies `stopped near the` as context; the target lists the missing spans with their identifying sentinels instead of repeating all visible text. S2 is the extra terminal sentinel in this chapter's paper-style convention. EOS ends the target. Shifting inserts BOS and omits that last EOS from the decoder input.

More generally, remove R words in K separated spans from N words. Then source length is $N - R + K + 1$, and target length is $R + K + 2$. The decoder input has the target's length. For just `(1, 4)`, $N = 9, R = 3, K = 1$: source length is eight, while target and decoder input lengths are six. The source is `the S0 near the old bridge today EOS`, the target is `S0 red car stopped S1 EOS`, and decoder input is `BOS S0 red car stopped S1`. Equal source and target lengths in the first example were a coincidence.

In [6]

```
span_sentence = encode("the red car stopped near the old bridge today")
for spans in ([(1, 3), (6, 8)], [(1, 4)]):
    source, decoder, target = span_corrupt(span_sentence, spans)
    removed = sum(end - start for start, end in spans)
    expected_source = len(span_sentence) - removed + len(spans) + 1
    expected_target = removed + len(spans) + 2
    assert (len(source), len(target), len(decoder)) == (expected_source, expected_target, expected_target)
    print("Spans:", spans)
    for name, row in (("Source", source), ("Target", target), ("Decoder input", decoder)):
        print(f"  {name:13s} ({len(row)}):", " ".join(token_names(row)))
```

Output

```
Spans: [(1, 3), (6, 8)]
  Source        (8): the S0 stopped near the S1 today EOS
  Target        (8): S0 red car S1 old bridge S2 EOS
  Decoder input (8): BOS S0 red car S1 old bridge S2
Spans: [(1, 4)]
  Source        (8): the S0 near the old bridge today EOS
  Target        (6): S0 red car stopped S1 EOS
  Decoder input (6): BOS S0 red car stopped S1
```

6. Check a loss before training

Zero logits give probability $1/V$ to every vocabulary item. Each evaluated label contributes $-\log(1/V) = \log V$. If m labels are selected, the sum is $m \log V$ and the mean is $\log V$.

Problem	**Selected labels**	**Loss sum with $V = 17$**	**Mean loss**
GPT continuation	3	8.499640	2.833213
BERT selected token	1	2.833213	2.833213
T5 reconstruction	8	22.665707	2.833213

Two appended IGNORE labels add neither terms nor a denominator count. Their input PAD token may still be one of the 17 output vocabulary classes at other evaluated positions. Ignoring a position does not remove a class from those distributions. Uniform probabilities are a useful baseline, not a maximum possible loss. This arithmetic check validates counting, not which family would be better after learning on a common downstream task.

In [7]

```
from examples.ch07_pretraining.run_examples import prepared_examples
for family, (_, labels) in prepared_examples().items():
    logits = torch.zeros(*labels.shape, len(VOCAB), dtype=torch.float64)
    mean = selected_loss(logits, labels)
    count = int((labels != IGNORE).sum())
    extended_labels = F.pad(labels, (0, 2), value=IGNORE)
    extended_logits = torch.zeros(*extended_labels.shape, len(VOCAB), dtype=torch.float64)
    torch.testing.assert_close(mean, selected_loss(extended_logits, extended_labels))
    print(f"{family.upper():4s}: labels={count}, sum={mean.item()*count:.6f}, mean={mean.item():.6f}; padding
     invariant")
```

Output

```
GPT : labels=3, sum=8.499640, mean=2.833213; padding invariant
BERT: labels=1, sum=2.833213, mean=2.833213; padding invariant
T5  : labels=8, sum=22.665707, mean=2.833213; padding invariant
```

7. Choose what changes

Reuse method	Stack parameters	New task-head parameters
Prompt and predict	Fixed	No new head is required
Train only a new head	Fixed	Updated by the supervised loss
Fine-tune stack and head	Updated	Updated

In the causal input `the red car stopped`, the first position can read only `the`. The last valid position can read the complete sentence, including `red`. Extra causal layers do not give the first position access to later keys. The numerical probe changes only red to blue; it leaves the first hidden vector fixed while the last vector can change.

For the chapter's head, multiply the row vector $h = (1, 2)$ by the two columns of W. The first score is $1(0.5) + 2(1) = 2.5$ and the second is $1(-0.5) + 2(0) = -0.5$. With zero bias the class probabilities are

$$\mathrm{softmax}(2.5, -0.5) \approx (0.952574, 0.047426).$$

These are two class probabilities, not a 17-token distribution. Finally, `eval()` changes behavior such as dropout; it does not switch off gradients or prohibit optimizer updates. Freeze parameters with `requires_grad_(False)` and exclude them from the optimizer. With stochastic layers, choose the frozen network's evaluation behavior separately. This teaching stack has zero dropout.

In [8]

```
head_example_h = torch.tensor([[1., 2.]], dtype=torch.float64)
head_example_w = torch.tensor([[.5, -.5], [1., 0.]], dtype=torch.float64)
print("Scores:", (head_example_h @ head_example_w).tolist())
print("Class probabilities:", (head_example_h @ head_example_w).softmax(-1).tolist())
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(70)
    pooling_model = TinyTokenModel(causal=True).double().eval()
pooling_ids = torch.tensor([encode("the red car stopped"), encode("the blue car stopped")])
with torch.no_grad():
    pooling_hidden = pooling_model.encode_hidden(pooling_ids)
first_delta = (pooling_hidden[0, 0] - pooling_hidden[1, 0]).abs().max().item()
last_delta = (pooling_hidden[0, -1] - pooling_hidden[1, -1]).abs().max().item()
assert first_delta == 0
print(f"Changed color: first-vector delta={first_delta:.6g}; last-vector delta={last_delta:.6g}")
```

Output

```
Scores: [[2.5, -0.5]]
Class probabilities: [[0.9525741268224334, 0.04742587317756679]]
Changed color: first-vector delta=0; last-vector delta=0.538734
```

8. Follow all three BERT treatments

Select Python index 1 in `the red car stopped`. The original label is always red, while one draw selects MASK (below 0.8), a random ordinary word (from 0.8 up to 0.9), or the unchanged input (at least 0.9). The canonical helper's `return_trace=True` reports the decision it already made. No extra random draw is consumed and no source file needs to be edited.

Create the generator once. Recreating a generator with seed 19 on every call would repeat its first treatment instead of exploring the mixture. A random replacement consumes an additional integer draw; the trace follows that actual random sequence.

In [9]

```
original = torch.tensor(encode("the red car stopped"))
selected_index = 1
original_id = original[selected_index].item()
unselected = torch.arange(original.numel()) != selected_index
corruption_rng = torch.Generator().manual_seed(19)
first_examples, same_word_example = {}, None
for trial in range(1, 257):
    inputs, labels, trace = bert_corrupt(original, [selected_index],
                                         corruption_rng, return_trace=True)
    decision = trace[0]
    assert labels[selected_index] == original_id
    assert torch.all(labels[unselected] == IGNORE)
    assert torch.equal(inputs[unselected], original[unselected])
    example = trial, decision, inputs.tolist(), labels.tolist()
    first_examples.setdefault(decision["treatment"], example)
    if decision["treatment"] == "random" and inputs[selected_index] == original_id:
        same_word_example = example
    if len(first_examples) == 3 and same_word_example is not None: break
assert set(first_examples) == {"mask", "random", "unchanged"}
assert same_word_example is not None
for trial, decision, inputs, labels in [*first_examples.values(), same_word_example]:
    print(f"Call {trial:3d}: draw={decision['draw']:.9f}, {decision['treatment']},
     input={VOCAB[decision['input_id']]}")
    print("  inputs", inputs, "labels", labels)
```

Output

```
Call   1: draw=0.968602955, unchanged, input=red
  inputs [7, 8, 9, 10] labels [-100, 8, -100, -100]
Call   2: draw=0.199866176, mask, input=[MASK]
  inputs [7, 3, 9, 10] labels [-100, 8, -100, -100]
Call   3: draw=0.879927933, random, input=the
  inputs [7, 7, 9, 10] labels [-100, 8, -100, -100]
Call  93: draw=0.872130215, random, input=red
  inputs [7, 8, 9, 10] labels [-100, 8, -100, -100]
```

In the recorded run, calls 1-3 cover all three treatments. The mask changes ID 8 to ID 3 (`[MASK]`), and the first random replacement changes ID 8 to ID 7 (*the*). Call 1 and call 93 both leave the input IDs at `[7, 8, 9, 10]` with labels `[-100, 8, -100, -100]`. Call 1 used the unchanged branch because its draw was about 0.969. Call 93 used the random branch because its draw was about 0.872, and the additional integer draw happened to return ID 8.

The replacement range includes all ten ordinary-word IDs from 7 through 16, including the original. Given the random branch in this vocabulary, drawing *red* has probability $1/10$. The arrays alone cannot distinguish the two calls. The recorded draw identifies the branch, while the label records the original word in either case.

First illustrative call	Draw	Treatment	Supplied word	Reference label
1	0.968603	unchanged	red	red
2	0.199866	mask	MASK	red
3	0.879928	random	the	red
93	0.872130	random	red	red

9. Find where the direct loss acts

Imagine a table with four rows, one per input position, and one column for each of the $V = 17$ vocabulary items. A **logit** is the score in a cell of that table before applying softmax. We make a batch with one table, so the tensor has shape $[1, 4, V]$. The only graded label is *red* at Python index 1. The other three entries are `IGNORE`, so `selected_loss` excludes their rows from both the sum of token losses and the count used to calculate the mean.

Write $z_{i,k}$ for the logit at position i for vocabulary ID k in this one-example batch. The target ID at position 1 is $y = 8$. With a single graded position, the loss is

$$L = -z_{1,y} + \log \left(\sum_{k=0}^{V-1} \exp z_{1,k} \right).$$

Only row 1 appears in this expression. Its gradient is the row's softmax probability vector minus a vector that is 1 at ID y and 0 everywhere else. To see why, differentiate the log of the sum. At ID k the derivative is $\exp z_{1,k}$ divided by the sum of all V exponentials. Differentiating $-z_{1,y}$ subtracts 1 only at the correct ID. Every other row is absent from L, so its partial derivatives are exactly zero.

In [10]

```
from examples.ch07_pretraining.model import TinyTokenModel, selected_loss

vocab_size = len(VOCAB)
target_id = encode("red")[0]
loss_rng = torch.Generator().manual_seed(73)
random_logits = torch.randn(
    (1, 4, vocab_size), generator=loss_rng, requires_grad=True
)
loss_labels = torch.tensor([[IGNORE, target_id, IGNORE, IGNORE]])
direct_loss = selected_loss(random_logits, loss_labels)
direct_loss.backward()

direct_gradient = random_logits.grad[0]
expected_gradient = torch.zeros_like(direct_gradient)
expected_gradient[1] = random_logits.detach()[0, 1].softmax(dim=-1)
expected_gradient[1, target_id] -= 1
torch.testing.assert_close(direct_gradient, expected_gradient)
assert torch.count_nonzero(direct_gradient[[0, 2, 3]]) == 0

print(f"Loss: {direct_loss.item():.6f}; target ID: {target_id}")
for position, row in enumerate(direct_gradient):
    print(f"Position {position}: nonzero={row.count_nonzero().item():2d}, norm={row.norm().item():.6f}")
print("Selected-row gradient at correct ID:", direct_gradient[1, target_id].item())
```

Output

```
Loss: 3.428197; target ID: 8
Position 0: nonzero= 0, norm=0.000000
Position 1: nonzero=17, norm=1.017874
Position 2: nonzero= 0, norm=0.000000
Position 3: nonzero= 0, norm=0.000000
Selected-row gradient at correct ID: -0.9675546288490295
```

The gradient at the correct ID 8 in row 1 is negative. A small increase to that logit reduces the loss. The other 16 numbers are positive because increasing a competing logit takes probability away from the correct ID. Rows 0, 2, and 3 have 17 exact zeros each.

Now feed *the [MASK] car stopped* to the bidirectional teaching model and keep the same labels. The loss still has a direct gradient only at the second output. That output can attend to *the*, *car*, and *stopped*. Each of those

input vectors can affect the values or attention weights used to build the hidden vector at index 1. The chain rule propagates the loss along those paths back to the input vectors. The companion code saves the embedding lookup result so we can inspect gradients by input **position**, even if a token ID appears more than once in a sentence.

In [11]

```
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(74)
    gradient_model = TinyTokenModel(causal=False).eval()

lookup_outputs = []

def save_embedding_output(module, arguments, output):
    output.retain_grad()
    lookup_outputs.append(output)

masked_ids = encode("the red car stopped")
masked_ids[1] = MASK
gradient_input = torch.tensor([masked_ids])
with gradient_model.embedding.register_forward_hook(save_embedding_output):
    model_logits = gradient_model(gradient_input)
    model_logits.retain_grad()
    selected_loss(model_logits, loss_labels).backward()

output_norms = model_logits.grad[0].norm(dim=-1)
input_norms = lookup_outputs[0].grad[0].norm(dim=-1)
assert torch.count_nonzero(output_norms[[0, 2, 3]]) == 0
assert torch.all(input_norms[[0, 2, 3]] > 0)
print(f"{'pos':>3}  {'input':7s}  {'logit grad norm':>16s}"
      f"  {'lookup grad norm':>16s}")
for index, name in enumerate(token_names(gradient_input[0])):
    print(f"{index:3d}  {name:7s}  {output_norms[index]:16.8f}"
          f"  {input_norms[index]:16.8f}")
```

Output

```
pos  input     logit grad norm  lookup grad norm
  0  the            0.00000000        0.08548552
  1  [MASK]         1.01604807        0.63174695
  2  car            0.00000000        0.19065118
  3  stopped        0.00000000        0.05139231
```

In this run the three ungraded positions have nonzero input-vector gradient norms, even though their output-gradient norms are zero. The lookup-vector gradient at a position comes from all graded outputs whose computations depend on it. When the model updates its embedding table, gradients from repeated uses of the same ID are added into that ID's shared row. An allowed path can carry a zero or small gradient for particular weights or inputs. The experiment exhibits a path that actually carries a signal here.

The measured gradients make the distinction visible:

Position	Supplied input	Direct logit-gradient norm	Input-vector gradient norm
0	the	0	0.085486
1	MASK	1.016048	0.631747
2	car	0	0.190651
3	stopped	0	0.051392

10. Test a future-token boundary

First run the book's `visibility_probe` in `examples/ch07_pretraining/run_examples.py`. It compares *the red car stopped* with *the red car moved* at Python index 1 (*red*). Then change the shared color to *blue* and repeat the comparison at the same index. The only difference **within each pair** is at index 3. The two stacks have the same parameters and buffers. They differ in their attention permission rule, so any difference in the measured sensitivity arises from that rule.

Compare the two whole vocabulary vectors **within** each stack. Our summary is the maximum absolute coordinate change over all V logits. With CPU `float64` computation, use an absolute tolerance of 10^{-12} and zero relative tolerance for the causal check. We also print the two changed-word logits as examples of individual coordinates in the comparison. Both models are in evaluation mode and this probe requires no gradients.

In [12]

```
from examples.ch07_pretraining.run_examples import visibility_probe

tolerance = 1e-12
with torch.random.fork_rng(devices=[]):
    canonical = visibility_probe()
    torch.manual_seed(71)
    causal_stack = TinyTokenModel(causal=True).double().eval()
    bidirectional_stack = TinyTokenModel(causal=False).double().eval()
    bidirectional_stack.load_state_dict(causal_stack.state_dict())

print("Canonical pair at red")
for stack_name, difference in canonical.items():
    print(f"  {stack_name:13s} max |change| = {difference:.12g}")
assert canonical["causal"] <= tolerance

blue_before = torch.tensor([encode("the blue car stopped")])
blue_after = torch.tensor([encode("the blue car moved")])
blue_index = 1
report_ids = encode("stopped moved")
assert torch.where(blue_before[0] != blue_after[0])[0].tolist() == [3]
```

Output

```
Canonical pair at red
  causal        max |change| = 0
  bidirectional max |change| = 0.195011679533
```

In [13]

```
print("New pair at blue")
for stack_name, stack in (
    ("causal", causal_stack), ("bidirectional", bidirectional_stack)
):
    with torch.no_grad():
        before = stack(blue_before)[0, blue_index]
        after = stack(blue_after)[0, blue_index]
    max_difference = (after - before).abs().max().item()
    print(f"  {stack_name:13s} max |change| = {max_difference:.12g}")
    for word in report_ids:
        print(f"    {VOCAB[word]:7s} {before[word]:+.8f}"
              f" -> {after[word]:+.8f}")
    if stack_name == "causal":
        torch.testing.assert_close(before, after, atol=tolerance, rtol=0)
```

Output

```
New pair at blue
  causal        max |change| = 0
    stopped -0.51536650 -> -0.51536650
    moved   -0.16878307 -> -0.16878307
  bidirectional max |change| = 0.168832161318
    stopped -0.45795604 -> -0.61841434
    moved   -0.01385238 -> -0.18268454
```

In both pairs the causal stack gives the same logits before and after the change at index 3. To see why this holds through multiple layers, start at the embeddings. The first two positions use the same token IDs and position vectors in the two inputs. At every causal layer a query at index i can read only keys at indices $j \leq i$. Thus no layer can move information from index 3 to index 1, directly or through an intermediate position. The residual connections, layer normalization, feed-forward network, and final projection act within each position, so they preserve this argument. The causal invariance follows from the mask and the other operations in this stack. The numeric tolerance only governs how we check the equality on a computer.

The bidirectional stack allows index 1 to read index 3, and the nonzero difference in each pair shows that the changed word reached the earlier logits with these weights. The same permission could yield a small or zero difference if a head gives that position very little weight, if the changed values project to the same features, or if later computations cancel the change. A nonzero difference is an observation for this particular probe.

Changed final word	Causal maximum logit change	Bidirectional maximum logit change
After the red car	0	0.195012
After the blue car	0	0.168832

The zeros test forbidden information paths; the nonzero values exhibit allowed paths with these particular weights.

11. Inspect a training run

From the repository root, run `python -m examples.ch07_pretraining.run_examples -train`. This invokes the three teaching models on one fixed training example per family for 200 Adam steps. We run the same module from the notebook using its Python interpreter and parse the printed JSON so that the table and token comparison come from the actual run.

`selected_labels` is the number of label slots graded in each fixed example. The reported NLL, or negative log-likelihood, is the mean of the token cross-entropies over those slots. In the BERT example only the masked *red* is graded. For GPT each of its three inputs supplies one next-token target. For T5 all eight decoder targets are graded, including the sentinels and `EOS`.

In [14]

```
import json
import subprocess

command = [
    sys.executable, "-m", "examples.ch07_pretraining.run_examples",
    "--train",
]
run_result = subprocess.run(
    command, cwd=repo_root, check=True, capture_output=True, text=True
)
report = json.loads(run_result.stdout)
checks = report["fixed_example_checks"]
expected_counts = {"gpt": 3, "bert": 1, "t5": 8}

print(f"{'Family':6s} {'Graded':>6s} {'Initial NLL':>12s}"
      f" {'Final NLL':>12s} {'Correct':>8s}")
for family, graded in expected_counts.items():
    result = checks[family]
    predicted = result["selected_predictions"]
    targets = result["selected_targets"]
    correct = sum(
        pred == target for pred, target in zip(predicted, targets)
    )
    assert result["steps"] == 200
    assert result["selected_labels"] == graded == len(targets)
    assert len(predicted) == graded
    assert result["correct_selected"] == correct == graded
    assert result["final_nll"] < result["initial_nll"]
    print(f"{family.upper():6s} {graded:6d} {result['initial_nll']:12.8f}"
          f" {result['final_nll']:12.8f} {correct:4d}/{graded:<3d}")
```

Output

```
Family Graded  Initial NLL    Final NLL  Correct
GPT         3   2.98642468   0.00116479    3/3
BERT        1   2.88157463   0.00049924    1/1
T5          8   2.89228392   0.00153717    8/8
```

In [15]

```
for family, result in checks.items():
    print(family.upper())
    print("  predictions", " ".join(result["selected_predictions"]))
    print("  targets    ", " ".join(result["selected_targets"]))
assert checks["t5"]["exact_generated"]
print("  generated  ", " ".join(checks["t5"]["generated"]))
```

Output

```
GPT
  predictions red car stopped
  targets     red car stopped
BERT
  predictions red
  targets     red
T5
  predictions S0 red car S1 old bridge S2 EOS
  targets     S0 red car S1 old bridge S2 EOS
  generated   S0 red car S1 old bridge S2 EOS
```

The independently rerun 200-update check produced:

Objective	Graded labels	Initial mean NLL	Final mean NLL	Correct labels
GPT	3	2.986425	0.001165	3/3
BERT	1	2.881575	0.000499	1/1
T5	8	2.892284	0.001537	8/8

GPT predicts *red*, *car*, and *stopped* in order, BERT predicts *red* at the mask, and T5 matches all eight tokens. For the T5 selected predictions, the decoder received the correct shifted target during the forward pass, a setup called **teacher forcing**. The run also reports a separate greedy generation check. It starts at `BOS`, feeds back each token it predicts, and reproduces the target through `EOS` for this same training source.

The loss drops and the targets match after optimization. Together these results check that the target shifts, masking, vocabulary projection, loss, gradient, and optimizer work together well enough to fit these three training examples. Repeating one fixed example lets a model memorize its targets. The run gives no held-out measurement, and the three means summarize different targets and different visible information. For example, BERT scores one original word with left and right context, while GPT scores three words using only their prefixes. A smaller training mean on one of these objectives does not imply better predictions on a common task.

For an initial held-out check, reserve *the blue car moved near the old bridge today*. Use its first four words for the two short objectives and the whole sentence for T5, mirroring the lengths used in the training demo. Reserve both forms before generating corruptions so none of their derived inputs is used for training. Keep the weights fixed during evaluation and use these examples.

Family	Held-out input	Graded target
GPT	`the blue car`	`blue car moved`
BERT	`the [MASK] car moved`	`blue` at index 1
T5	`the S0 moved near the S1 today EOS`	`S0 blue car S1 old bridge S2 EOS`

For T5 the teacher-forced decoder input is `BOS S0 blue car S1 old bridge S2`. Score the graded slots of each family on its own objective and separately check T5's freely generated sequence for an exact match. This is a new in-vocabulary sentence, although *blue* and *moved* did not appear in the demo's training examples. Its

score could reflect that lack of training exposure as well as the change of sentence. To isolate recombination in a larger experiment, expose the model to those words in several other training sentences while holding back their new combinations. Because continuations and masked words can be ambiguous, evaluate many held-out sentences and use the recorded targets rather than treating a single answer as the only grammatical possibility.

12. Check both sides of an encoder-decoder (optional)

Exercise. Use one small random encoder-decoder in evaluation mode. Compare decoder logits after changing a later decoder input, replacing a valid source token, and appending a masked source PAD. Write out the source validity masks.

Use the same span-corruption example as Exercise 5. Its decoder input begins with `BOS S0 red car S1 old`. At decoder index 1, the input is `S0` and the model predicts the next token, `red`. The causal self-attention mask lets that prediction use decoder inputs at indices 0 and 1. Cross-attention can also use every valid source key, including the source token `stopped` at index 2. Decoder index 1 and source index 2 belong to different sequences, so their index order places no restriction on cross-attention.

Keep one set of weights for all calls. Use CPU float64, evaluation mode, and disabled gradient recording. Compare whole vectors of vocabulary logits using the maximum absolute difference. With absolute tolerance `1e-12` and zero relative tolerance, two slices pass the invariance check if each corresponding entry differs by at most `1e-12`.

In [16]

```
from examples.ch07_pretraining.data import (
    PAD, TOKEN_TO_ID, encode, span_corrupt, token_names,
)
from examples.ch07_pretraining.model import make_model

torch.manual_seed(310)
t5_probe = make_model("t5", n_layers=2).double().eval()

source_ids, decoder_ids, _ = span_corrupt(
    encode("the red car stopped near the old bridge today"),
    [(1, 3), (6, 8)],
)
source_ids = source_ids[None]
decoder_ids = decoder_ids[None]

valid_source = torch.ones_like(source_ids, dtype=torch.bool)
valid_decoder = torch.ones_like(decoder_ids, dtype=torch.bool)
padded_source = F.pad(source_ids, (0, 1), value=PAD)
valid_padded_source = F.pad(valid_source, (0, 1), value=False)

print("Source tokens:", token_names(source_ids[0]))
print("Decoder input:", token_names(decoder_ids[0]))
print("Unpadded mask:", valid_source.tolist())
print("Padded mask:", valid_padded_source.tolist())
```

Output

```
Source tokens: ['the', 'S0', 'stopped', 'near', 'the', 'S1', 'today', 'EOS']
Decoder input: ['BOS', 'S0', 'red', 'car', 'S1', 'old', 'bridge', 'S2']
Unpadded mask: [[True, True, True, True, True, True, True, True]]
Padded mask: [[True, True, True, True, True, True, True, True, False]]
```

Replace `old` with `blue` at decoder index 5 and measure the logits at indices 0-4. At every decoder layer, each of those earlier positions reads only its own prefix, so there is no route from input index 5 to the earlier logits. Decoder row 5 can read its own input and provides a useful check that the replacement reached the model.

Next, change `stopped` to `moved` at source index 2 and hold the entire decoder input fixed. The source encoder can incorporate the replacement into any valid source row, and a decoder query can read the resulting source memory. Even logits at decoder index 0, where the decoder input is just `BOS` at that position, may change.

Finally, append a PAD on the right, preserving the original source indices. The `False` mask entry excludes the new position as a key in encoder self-attention and decoder cross-attention. Compare every decoder output, since the source padding restriction applies to every decoder query.

In [17]

```
later_position = 5
changed_decoder = decoder_ids.clone()
changed_decoder[0, later_position] = TOKEN_TO_ID["blue"]

changed_source = source_ids.clone()
changed_source[0, 2] = TOKEN_TO_ID["moved"]

with torch.inference_mode():
    base_logits = t5_probe(
        source_ids, decoder_ids, valid_source, valid_decoder
    )
    later_logits = t5_probe(
        source_ids, changed_decoder, valid_source, valid_decoder
    )
    source_logits = t5_probe(
        changed_source, decoder_ids, valid_source, valid_decoder
    )
    pad_logits, pad_weights = t5_probe(
        padded_source, decoder_ids, valid_padded_source,
        valid_decoder, return_weights=True,
    )
```

In [18]

```
def max_logit_delta(first, second):
    return (first - second).abs().max().item()


early = slice(0, later_position)
cases = [
    ("Later decoder edit, rows 0-4",
     base_logits[:, early], later_logits[:, early]),
    ("Later decoder edit, row 5",
     base_logits[:, 5], later_logits[:, 5]),
    ("Valid source edit, decoder row 0",
     base_logits[:, 0], source_logits[:, 0]),
    ("Masked source PAD, all decoder rows",
     base_logits, pad_logits),
]
atol = 1e-12
for description, first, second in cases:
    print(f"{description:<36} {max_logit_delta(first, second):.3e}")

torch.testing.assert_close(
    base_logits[:, early], later_logits[:, early],
    atol=atol, rtol=0,
)
torch.testing.assert_close(base_logits, pad_logits, atol=atol, rtol=0)
for attention_path in ("encoder", "cross"):
    assert all(
        layer[..., -1].count_nonzero().item() == 0
        for layer in pad_weights[attention_path]
    )
    print(f"{attention_path}: PAD key weight is zero in every layer")
```

Output

```
Later decoder edit, rows 0-4         0.000e+00
Later decoder edit, row 5            1.210e+00
Valid source edit, decoder row 0     6.912e-02
Masked source PAD, all decoder rows  0.000e+00
encoder: PAD key weight is zero in every layer
cross: PAD key weight is zero in every layer
```

The saved run reports zero difference for decoder rows 0-4 after the later decoder edit. At the edited row 5, at least one logit changes by about 1.21. Replacing the valid source token changes a row-0 logit by about 0.0691. The mask guarantees the earlier-row invariance. In the two other cases, the path is available and the size of the difference depends on the weights and inputs.

The masked PAD has zero key weight in both attention paths. The extra source position can still act as a query and produce a hidden vector. That vector cannot influence valid source rows or decoder rows because the key mask excludes it at each attention layer. Appending PAD leaves the original sinusoidal position indices unchanged. Floating-point kernels can round differently after a shape change, which is why the logit comparison uses a tolerance.

Intervention	Compared decoder logits	Maximum change
Replace later decoder input	Earlier rows 0-4	0
Replace later decoder input	The edited row 5	1.210
Replace a valid source word	Decoder row 0	0.06912
Append a masked source PAD	All decoder rows	0

The unpadded source mask has eight True entries; the padded version appends False. All encoder and cross-attention layers assign the added PAD key zero weight.

13. Add a task head (optional)

Exercise. Adapt the causal token model to classify a sentence as `red` (class 0) or `blue` (class 1). Compare training just a new classification head with updating the entire classifier. Use the same starting weights in both conditions and reserve a test split for the selected models.

Start from a color-free sentence template such as "today the {color} car stopped." Substituting `red` and `blue` produces a *counterfactual pair*: only the color word changes. Give every template both labels and assign each entire pair to one split. This balances sentence length, the color's position, and all non-color words across labels within each split. Validation and test use sentence arrangements held out from training. The color words themselves appear in training because the task is to recognize those words in new arrangements.

In [19]

```
from copy import deepcopy

from examples.ch07_pretraining.data import EOS
from examples.ch07_pretraining.model import TinyTokenModel

train_templates = (
    "the {color} car stopped",
    "the {color} car moved",
    "today the {color} car stopped",
    "today the {color} car moved",
    "the {color} car stopped today",
    "the {color} car moved today",
    "the {color} car stopped near the bridge",
    "the {color} car moved near the bridge",
    "the car stopped near the {color} bridge",
    "the car moved near the {color} bridge",
    "near the {color} bridge the car stopped",
    "near the {color} bridge the car moved",
)
validation_templates = (
    "the {color} car stopped near the old bridge",
    "today the {color} car moved near the bridge",
    "the car stopped near the {color} bridge today",
    "near the old {color} bridge the car moved",
)
test_templates = (
    "the old {color} car moved",
    "today the car stopped near the {color} bridge",
    "the {color} car moved near the old bridge today",
    "near the {color} bridge today the car stopped",
    "the car moved near the old {color} bridge",
    "near the old bridge the {color} car stopped",
)

split_templates = (train_templates, validation_templates,
                   test_templates)
all_templates = sum(split_templates, ())
assert len(set(all_templates)) == len(all_templates)
assert all(text.count("{color}") == 1 for text in all_templates)
```

Append `EOS` after the sentence, then right-pad the sequences within each batch. The colors stay in the input as ordinary vocabulary IDs. The supervised target is a separate class index per sentence. It never enters the token

stack. Because each pair uses an identical template, a rule based only on words such as `car` or `bridge` must get exactly one of the two labels right.

The paired split contains 24 training sentences (12 pairs), eight validation sentences (four pairs), and 12 test sentences (six pairs). Every split has equal numbers of red and blue examples. The exact templates are listed in the Python edition so the experiment can be reproduced.

In [20]

```
class_names = ("red", "blue")
color_ids = {TOKEN_TO_ID[name] for name in class_names}


def color_batch(templates):
    sentences, rows, labels = [], [], []
    for template in templates:
        for label, color in enumerate(class_names):
            sentence = template.format(color=color)
            row = encode(sentence) + [EOS]
            assert sum(token in color_ids for token in row) == 1
            sentences.append(sentence)
            rows.append(torch.tensor(row))
            labels.append(label)
    ids = nn.utils.rnn.pad_sequence(
        rows, batch_first=True, padding_value=PAD
    )
    return ids, torch.tensor(labels), sentences


train_ids, train_labels, _ = color_batch(train_templates)
valid_ids, valid_labels, _ = color_batch(validation_templates)
print("Training class counts:", train_labels.bincount().tolist())
print("Validation class counts:", valid_labels.bincount().tolist())
```

Output

```
Training class counts: [12, 12]
Validation class counts: [4, 4]
```

The chapter's token model already computes a final vector of length d_{model} at every position. Its usual vocabulary projection maps that vector to 17 token scores. Read `encode_hidden(ids)` before the vocabulary projection. The classifier replaces its unused vocabulary projection with `nn.Identity()` to remove parameters it does not need. The hidden-vector method returns representations with shape $[B, T, d_{\text{model}}]$ while retaining the embeddings, positions, causal mask, and layers. Here B is the number of sentences in the batch and T is the number of positions in each padded row.

For a row with L valid positions, the last valid index is $L - 1$ because positions start at zero and padding is on the right. That index contains the appended `EOS`. Its causal prefix includes the whole sentence, even when the color occurs near the end. Index each row by its own length so that pooling never selects PAD. A linear map on the resulting vector outputs two scores per sentence. Cross-entropy uses those logits and the integer class index, and the predicted class is the index of the larger logit.

In [21]

```
class ColorClassifier(nn.Module):
    def __init__(self, backbone):
        super().__init__()
        self.backbone = backbone
        self.backbone.vocabulary_projection = nn.Identity()
        self.head = nn.Linear(backbone.d_model, len(class_names))

    def forward(self, ids):
        hidden = self.backbone.encode_hidden(ids)
        last_valid = (ids != PAD).sum(dim=1) - 1
        batch_indices = torch.arange(ids.shape[0], device=ids.device)
        sentence_vectors = hidden[batch_indices, last_valid]
        return self.head(sentence_vectors)


torch.manual_seed(311)
color_base = ColorClassifier(TinyTokenModel(causal=True))
example_logits = color_base(train_ids[:2])
print("One two-logit row per sentence:", list(example_logits.shape))
print("Trainable parameters if the stack is frozen:",
      sum(p.numel() for p in color_base.head.parameters()))
print("Trainable parameters if the stack is updated:",
      sum(p.numel() for p in color_base.parameters()))
```

Output

```
One two-logit row per sentence: [2, 2]
Trainable parameters if the stack is frozen: 34
Trainable parameters if the stack is updated: 4626
```

For each training condition and learning rate, copy `color_base`, including its randomly initialized head. A deep copy gives each trial its own parameter tensors. Freeze the backbone in the head-only trials and pass only parameters that require gradients to a fresh Adam optimizer. In the full-network trials the trainable backbone can change how it encodes the sentence. The classifier always updates its head.

Use the same full training batch in every update and run each candidate for 80 updates. Try learning rates 0.003 and 0.01 under the same rules. Record validation mean cross-entropy (negative log likelihood, or NLL) at initialization and after every 10 updates. Keep the lowest validation NLL checkpoint for each rate, breaking exact ties in favor of the earlier update. Then select the rate with the lowest validation NLL, breaking rate ties in favor of the smaller rate. Thus the budget in data exposure and updates matches across conditions. Backpropagating through the whole backbone costs more computation per update.

In [22]

```
def fit_color_trial(freeze, learning_rate):
    model = deepcopy(color_base)
    model.backbone.requires_grad_(not freeze)
    optimizer = torch.optim.Adam(
        [p for p in model.parameters() if p.requires_grad],
        lr=learning_rate,
    )
    best = None
    for step in range(81):
        if step > 0:
            model.train()
            optimizer.zero_grad(set_to_none=True)
            loss = F.cross_entropy(model(train_ids), train_labels)
            loss.backward()
            optimizer.step()
        if step % 10 == 0:
```

```
            model.eval()
            with torch.no_grad():
                validation_nll = F.cross_entropy(
                    model(valid_ids), valid_labels
                ).item()
            if best is None or validation_nll < best[0]:
                best = validation_nll, step, deepcopy(model)
    return best


selected_trials = {}
for mode, freeze in (("head only", True), ("whole network", False)):
    candidates = []
    for learning_rate in (0.003, 0.01):
        val_nll, step, model = fit_color_trial(freeze, learning_rate)
        candidates.append((val_nll, learning_rate, step, model))
        print(f"{mode:<14} lr={learning_rate:.3f}  step={step:>2}  "
              f"validation NLL={val_nll:.6f}")
    selected_trials[mode] = min(
        candidates, key=lambda result: (result[0], result[1])
    )
```

Output

```
head only      lr=0.003  step=80  validation NLL=0.675192
head only      lr=0.010  step=30  validation NLL=0.674488
whole network  lr=0.003  step=80  validation NLL=0.002391
whole network  lr=0.010  step=80  validation NLL=0.000779
```

Check that freezing worked by measuring the largest change in any backbone parameter from initialization to the selected checkpoint. Then evaluate both selected models on the same held-out test batch. Report the number of correctly classified sentences and the number of pairs in which *both* colors are classified correctly. Pair accuracy exposes a model that gives the same answer when we swap the color. A fixed-class predictor scores half of the sentences and zero complete pairs.

In [23]

```
test_ids, test_labels, test_sentences = color_batch(test_templates)
print("Test class counts:", test_labels.bincount().tolist())

for mode, (_, rate, step, model) in selected_trials.items():
    backbone_delta = max(
        (now.detach() - start.detach()).abs().max().item()
        for now, start in zip(model.backbone.parameters(),
                              color_base.backbone.parameters())
    )
    if mode == "head only":
        assert backbone_delta == 0.0

    model.eval()
    with torch.inference_mode():
        logits = model(test_ids)
        predictions = logits.argmax(dim=-1)
        test_nll = F.cross_entropy(logits, test_labels).item()
    correct = predictions == test_labels
    complete_pairs = correct.reshape(-1, 2).all(dim=1)

    print(f"\n{mode}: lr={rate:.3f}, step={step}")
    print(f"  Maximum backbone change: {backbone_delta:.3e}")
    print(f"  Test NLL: {test_nll:.6f}")
    n_correct = correct.sum().item()
    print(f"  Correct sentences: {n_correct}/{len(correct)}")
```

```
    print(f"  Complete pairs: {complete_pairs.sum().item()}/"
          f"{len(complete_pairs)}")
    for sentence, actual, predicted in zip(
        test_sentences[:2], test_labels[:2], predictions[:2]
    ):
        print(f"  {sentence}")
        print(f"    target={class_names[actual]}, "
              f"prediction={class_names[predicted]}")
```

Output

```
Test class counts: [6, 6]

head only: lr=0.010, step=30
  Maximum backbone change: 0.000e+00
  Test NLL: 0.665279
  Correct sentences: 7/12
  Complete pairs: 1/6
  the old red car moved
    target=red, prediction=red
  the old blue car moved
    target=blue, prediction=blue

whole network: lr=0.010, step=80
  Maximum backbone change: 3.558e-01
  Test NLL: 0.000778
  Correct sentences: 12/12
  Complete pairs: 6/6
  the old red car moved
    target=red, prediction=red
  the old blue car moved
    target=blue, prediction=blue
```

Validation picks a learning rate of 0.01 in both conditions, but the frozen model's best checkpoint occurs at step 30 and the full-network model's best occurs at step 80. On the test split, the frozen model scores 7 out of 12 sentences and gets both members right in 1 of the 6 pairs. The full-network model scores 12 out of 12 sentences and gets all 6 pairs right. The two metrics describe different things. Getting 7 sentences right is one more success than always predicting a single label. Getting a whole pair right requires the model to respond correctly to a change in the color word while every other word stays fixed.

Both backbones start from random weights in this run. A frozen random backbone can already preserve some information about token identity through its embeddings and attention. Its head can learn a linear rule only from the features that reach the pooled `EOS` vector. Updating the backbone also lets the supervised signal change those features. With one seed and six test pairs, the observed difference describes this toy task and these settings.

To study **transfer**, first train a `TinyTokenModel` with the next-token objective on separate unlabeled text. Exclude all downstream sentence templates from that corpus and select the pretraining checkpoint with a separate language-model validation split. Load that checkpoint into a single backbone before replacing the vocabulary projection with the identity and initializing the two-logit task head. Copy that classifier for both adaptation regimes, give each trial a fresh optimizer, and run the same downstream validation and test protocol. Including the two random-start conditions alongside those two pretrained conditions lets you estimate the effect of pretraining at each level of adaptation.

Selected adaptation	Learning rate	Update	Validation NLL	Test NLL	Correct sentences	Complete pairs
Head only	0.01	30	0.674488	0.665279	7/12	1/6
Whole network	0.01	80	0.000779	0.000778	12/12	6/6

The frozen stack's maximum parameter change is exactly zero. The selected whole-network stack changes by about 0.356 in its largest coordinate. These are results from random initial weights, not a measurement of pretrained transfer.

14. Design a fair comparison (optional)

Use the color-classification task as a concrete design. Every sentence contains exactly one red or blue token, and non-color words vary independently of the label. Assign whole red/blue template pairs to training, validation, or test before creating any model-specific examples. Reserve distinct arrangements for each split. If the task is scaled to natural documents, also keep related documents or authors together to avoid near-duplicate leakage.

Design choice	Common rule and model-specific details
Data	Identical labeled sentences and split assignments for all systems; record sizes and class balance.
GPT interface	Causal stack; pool its last valid position and train a two-class head.
BERT interface	Bidirectional stack; use a declared pooling rule and a two-class head.
T5 interface	Prefix the source with the same task instruction and generate the class name followed by EOS.
Interpretation	Map class 0 to red and class 1 to blue for the heads. Require an exact allowed class string for generation; count malformed or missing answers as errors.
Adaptation	Declare whether all weights or only a head are trainable; apply the same chosen regime where comparable. Record exceptions.
Selection	Use the same validation split and a declared accuracy-based rule; break ties by a fixed rule such as lower cost and then earlier update.
Budget	Set a shared measured compute ceiling and tuning budget; report examples, updates, and elapsed training and inference time. Equal updates alone do not mean equal compute.
Final metric	Sentence accuracy, complete-pair accuracy, and the number of evaluated examples; evaluate each selected model once on test.

For a controlled study of architecture, use comparable small models and the same permitted pretraining corpus, while reporting parameter counts and compute. For a comparison of available pretrained checkpoints, disclose their different sizes, tokenizers, pretraining data, and training histories; the conclusion concerns those complete systems. These are different research questions, so specify which one the experiment answers.

Freeze the protocol before the final test. Tune learning rates and checkpoints only with validation results. Repeat the preregistered training seeds rather than selecting a lucky seed from test performance, and report variation along with per-example or per-pair outcomes. Do not turn a surprising test result into another round of tuning on that same test set.

Raw pretraining losses cannot identify the winner: they grade different tokens, reveal different context, use different corruption rules, and may use different vocabularies. A mean per selected token also hides how many positions supplied supervision. Lower training loss can reflect memorization. The relevant evidence is performance on the same held-out task under the declared data and resource constraints.